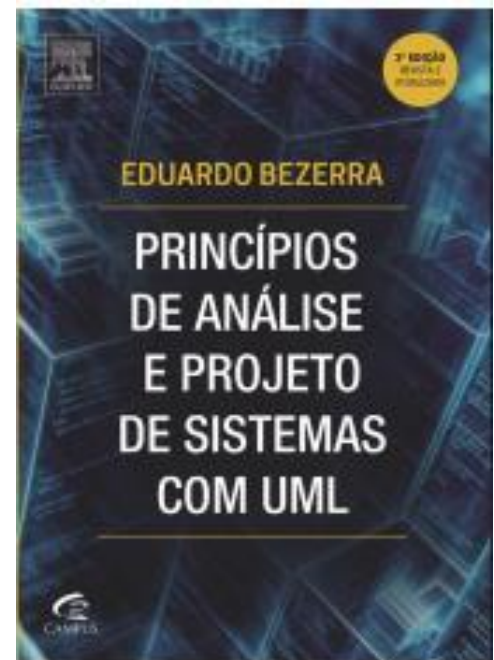


Princípios de Análise e Projeto de Sistemas com UML

3ª edição (2015)

Eduardo Bezerra
Editora Campus/Elsevier

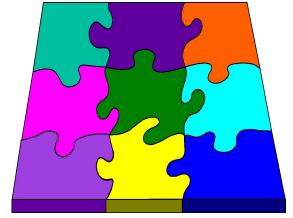


Capítulo 5

Modelagem de Classes de Análise

“O engenheiro de software amador está sempre à procura da mágica, de algum método sensacional ou ferramenta cuja aplicação promete tornar trivial o desenvolvimento de software. É uma característica do engenheiro de software profissional saber que tal panacéia não existe” -Grady Booch

Tópicos



- Introdução
- Diagrama de classes
- Diagrama de objetos
- Técnicas para identificação de classes
- Construção do modelo de classes
- Modelo de classes no processo de desenvolvimento

Introdução

- As funcionalidades de um SSOO é são realizadas internamente através de *colaborações* entre objetos.
 - Externamente, os atores visualizam resultados de cálculos, relatórios produzidos, confirmações de requisições realizadas, etc.
 - Internamente, os objetos colaboram uns com os outros para produzir os resultados.
- Essa colaboração pode ser vista sob o *aspecto dinâmico* e sob o *aspecto estrutural estático*.
- O **modelo de objetos** representa o aspecto estrutural e estático dos objetos que compõem um SSOO.
- Dois diagramas da UML são usados na construção do modelo de objetos:
 - **diagrama de classes**
 - **diagrama de objetos**

Introdução

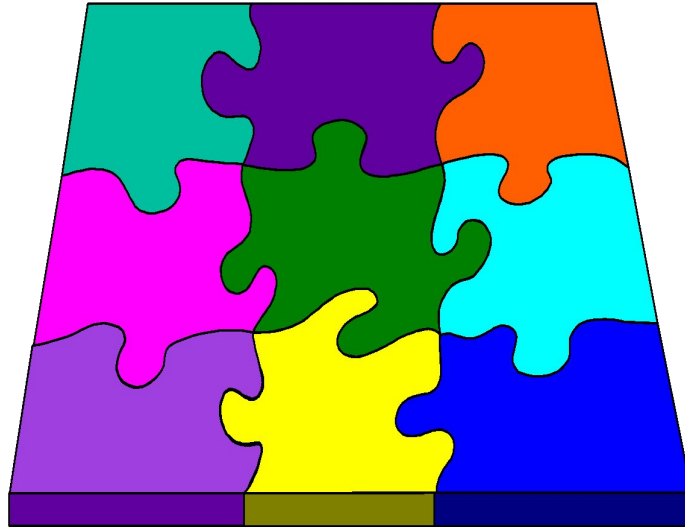
- Na prática o diagrama de classes é bem mais utilizado que o diagrama de objetos.
 - Tanto que o modelo de objetos é também conhecido como modelo de classes.
- Esse modelo *evolui* durante o desenvolvimento do SSOO.
 - À medida que o SSOO é desenvolvido, o modelo de objetos é incrementado com novos detalhes.
- Há três níveis sucessivos de detalhamento:
 - *Análise* → *Especificação (Projeto)* → *Implementação*.

Objetivo da Modelagem de Classes

- O objetivo da modelagem de classes de análise é prover respostas para as seguintes perguntas:
 - Por definição um sistema OO é composto de objetos...em um nível alto de abstração, que objetos constituem o sistema em questão?
 - Quais são as classes candidatas?
 - Como as classes do sistema estão relacionadas entre si?
 - Quais são as responsabilidades de cada classe?

Modelo de Classes de Análise

- Representa termos do domínio do negócio.
 - idéias, coisas, e conceitos no mundo real.
- Objetivo: descrever o problema representado pelo sistema a ser desenvolvido, sem considerar características da solução a ser utilizada.
- É um dicionário “visual” de conceitos e informações relevantes ao sistema sendo desenvolvido.
- Duas etapas:
 - modelo conceitual (modelo de domínio).
 - modelo da aplicação.
- Elementos de notação do diagrama de classes normalmente usados na construção do modelo de análise:
 - classes e atributos; associações, composições e agregações (com seus adornos); classes de associação; generalizações (herança).



5.2 Diagrama de classes

Classes

- Uma classe descreve esses objetos através de *atributos* e *operações*.
 - Atributos correspondem às informações que um objeto armazena.
 - Operações correspondem às ações que um objeto sabe realizar.
- Notação na UML: “caixa” com no máximo três compartimentos exibidos.
 - Detalhamento utilizado depende do estágio de desenvolvimento e do nível de abstração desejado.



Exemplo (classe ContaBancária)

ContaBancária

ContaBancária
número
saldo
dataAbertura

ContaBancária
criar()
bloquear()
desbloquear()
creditar()
debitar()

ContaBancária
número
saldo
dataAbertura
criar()
bloquear()
desbloquear()
creditar()
debitar()

ContaBancária
-número : String
-saldo : Quantia
-dataAbertura : Date
+criar()
+bloquear()
+desbloquear()
+creditar(in valor : Quantia)
+debitar(in valor : Quantia)

Associações

- Para representar o fato de que objetos podem se relacionar uns com os outros, utilizamos **associações**.
- Uma associação representa relacionamentos (ligações) que são formados entre objetos durante a execução do sistema.
- Note que, embora as associações sejam representadas entre classes do diagrama, tais associações representam ligações possíveis entre os objetos das classes envolvidas.

Notação para Associações

- Na UML associações são representadas por uma linha que liga as classes cujos objetos se relacionam.
- Exemplos:



Multiplicidades

- Representam a informação dos limites inferior e superior da quantidade de objetos aos quais outro objeto pode se associar.
- Cada associação em um diagrama de classes possui duas multiplicidades, uma em cada extremo da linha de associação.

Nome	Simbologia na UML
Apenas Um	1..1 (ou 1)
Zero ou Muitos	0..* (ou *)
Um ou Muitos	1..*
Zero ou Um	0..1
Intervalo Específico	$l_i \dots l_s$

Exemplos (multiplicidade)

- Exemplo 
 - Pode haver um cliente que esteja associado a vários pedidos.
 - Pode haver um cliente que não esteja associado a pedido algum.
 - Um pedido está associado a um, e somente um, cliente.

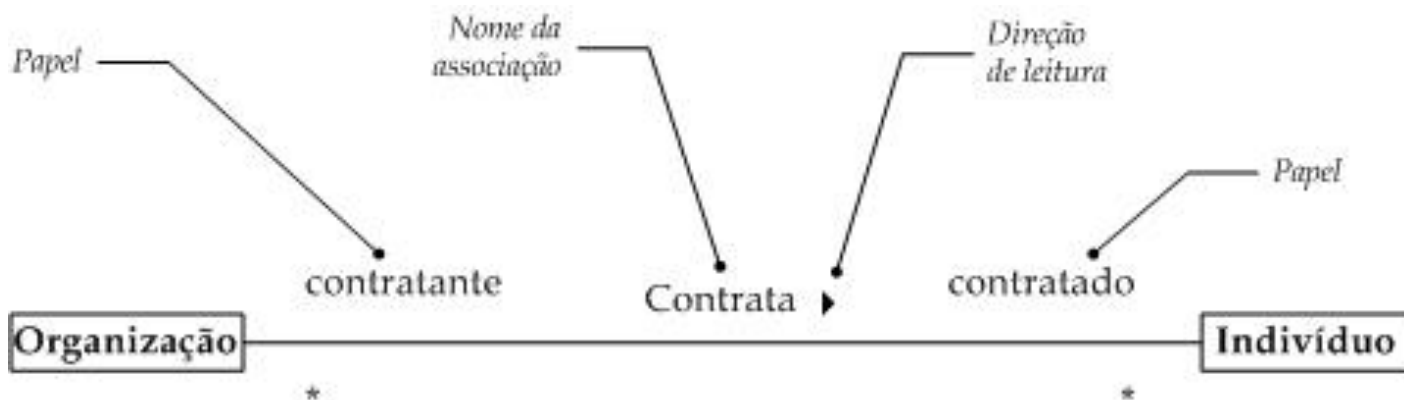
- Exemplo 
 - Uma corrida está associada a, no mínimo, dois velocistas
 - Uma corrida está associada a, no máximo, seis velocistas.
 - Um velocista *pode* estar associado a várias corridas.

Participação

- Uma característica de uma associação que indica a necessidade (ou não) da existência desta associação entre objetos.
- A participação pode ser *obrigatória* ou *opcional*.
 - Se o valor mínimo da multiplicidade de uma associação é igual a 1 (um), significa que a participação é obrigatória
 - Caso contrário, a participação é opcional.

Acessórios para Associações

- Para melhor esclarecer o significado de uma associação no diagrama de classes, a UML define três recursos de notação:
 - ***Nome da associação***: fornece algum significado semântico a mesma.
 - ***Direção de leitura***: indica como a associação deve ser lida
 - ***Papel***: para representar um papel específico em uma associação.

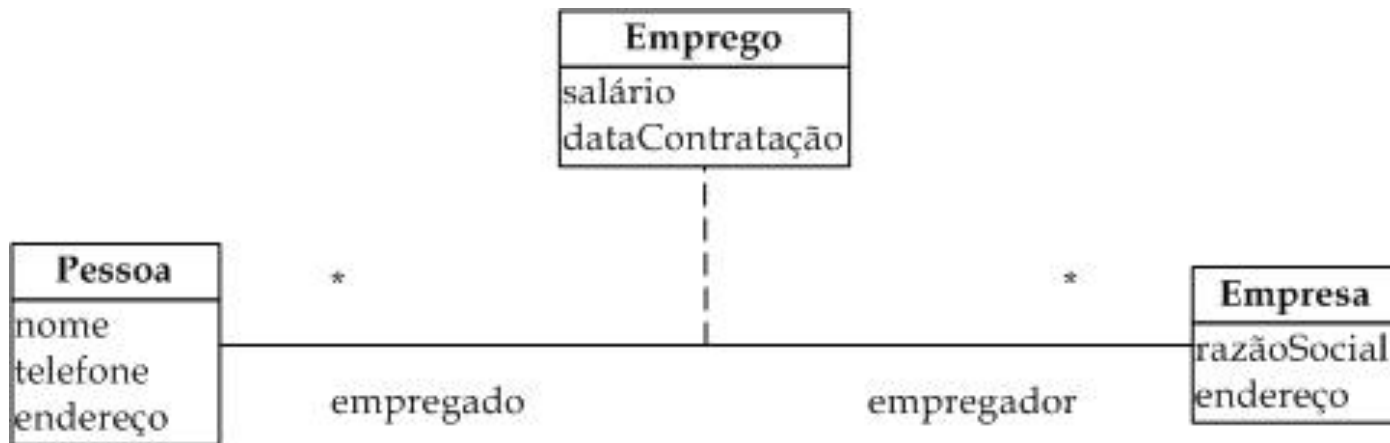


Classe associativa

- É uma classe que está ligada a uma associação, em vez de estar ligada a outras classes.
- É normalmente necessária quando duas ou mais classes estão associadas, e é necessário manter informações sobre esta associação.
- Uma classe associativa pode estar ligada a associações de qualquer tipo de conectividade.
- Sinônimo: *classe de associação*

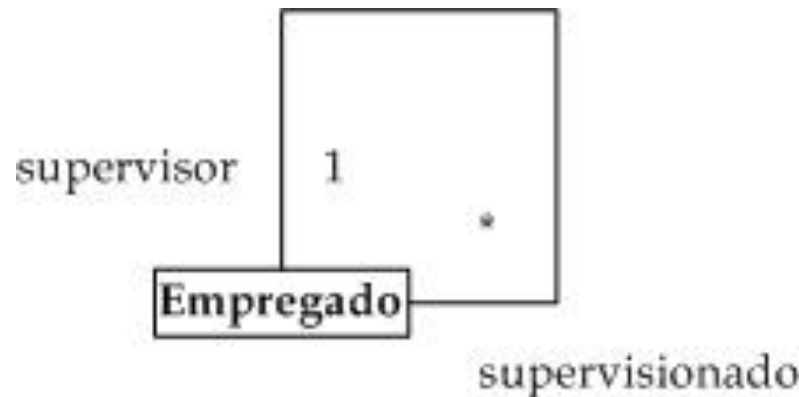
Notação para Classes Associativas

- Notação é semelhante à utilizada para classes ordinárias. A diferença é que esta classe é ligada a uma associação por uma linha tracejada.
- Exemplo: para cada par de objetos [pessoa, empresa], há duas informações associadas: salário e data de contratação.



Associações reflexivas

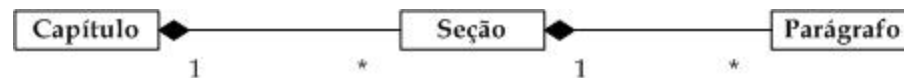
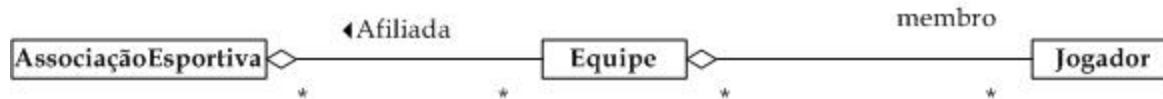
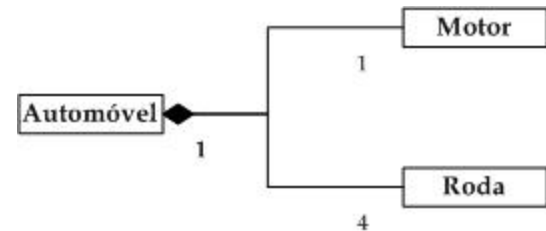
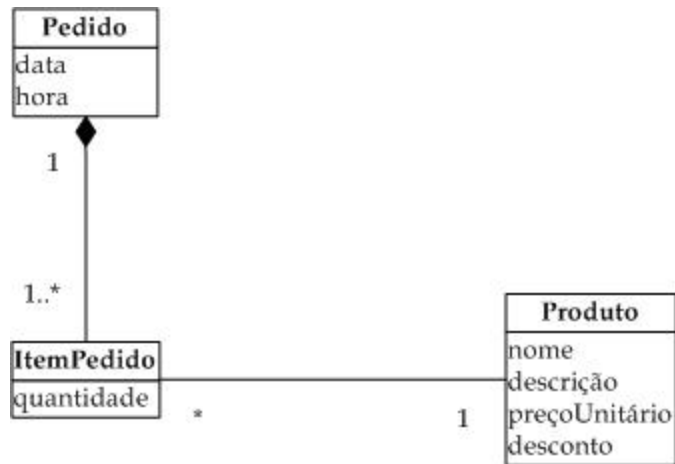
- Tipo especial de associação que representa ligações entre objetos que pertencem a uma mesma classe.
 - *Não* indica que um objeto se associa a ele próprio.
- Quando se usa associações reflexivas, a definição de papéis é importante para evitar ambigüidades na leitura da associação.
 - Cada objeto tem um papel distinto na associação.



Agregações e Composições

- A *semântica* de uma associação corresponde ao seu significado, ou seja, à natureza conceitual da relação que existe entre os objetos que participam daquela associação.
- De todos os significados diferentes que uma associação pode ter, há uma categoria especial de significados, que representa *relações todo-parte*.
- Uma relação todo-parte entre dois objetos indica que um dos objetos está contido no outro. Podemos também dizer que um objeto contém o outro.
- A UML define dois tipos de relacionamentos todo-parte, a *agregação* e a *composição*.

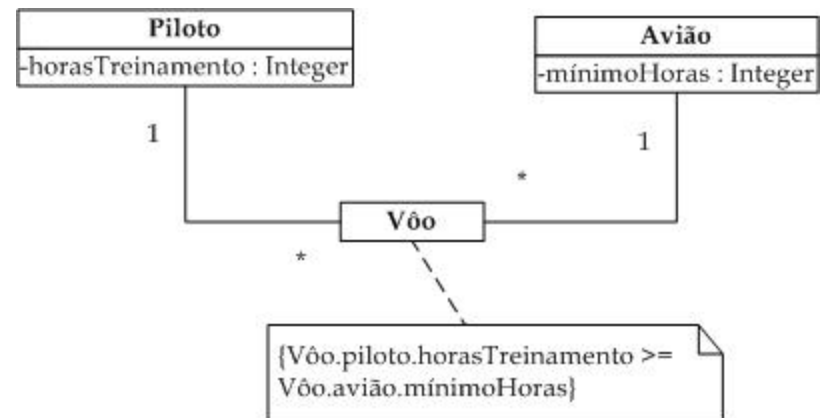
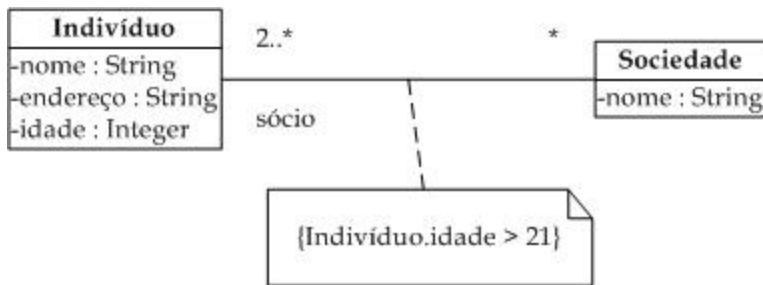
Exemplos



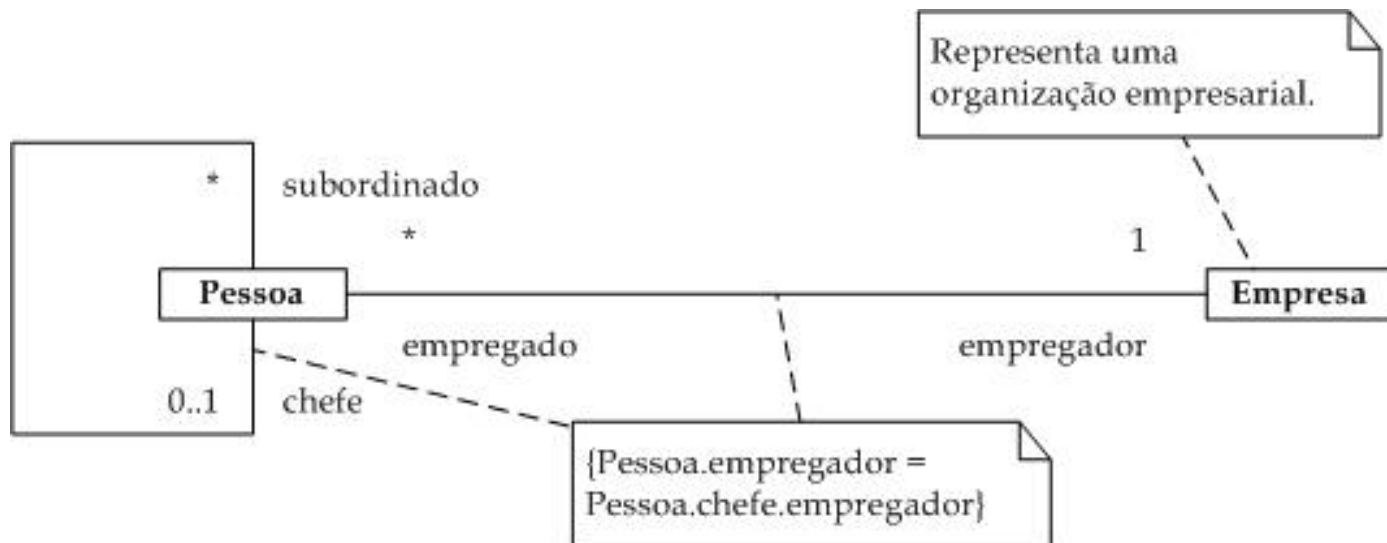
Agregações e Composições

- As diferenças entre a agregação e composição não são bem definidas. A seguir, as diferenças mais marcantes entre elas.
- Destruição de objetos
 - Na agregação, a destruição de um objeto todo não implica necessariamente na destruição do objeto parte.

Restrições sobre associações (cont)



Restrições sobre associações (cont)

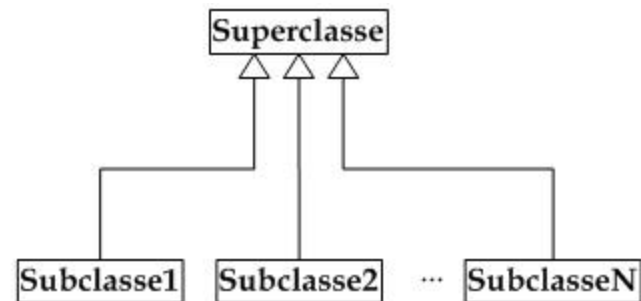
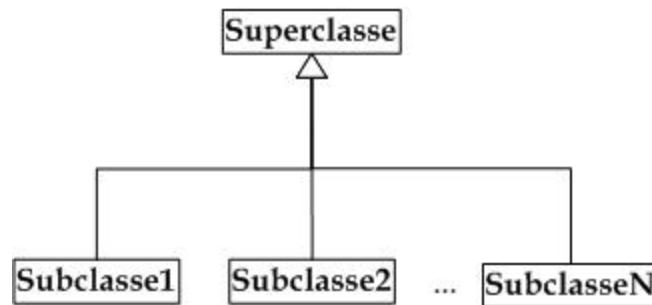


Generalizações e Especializações

- O modelador também pode representar relacionamentos entre classes.
 - Esses denotam relações de generalidade ou especificidade entre as classes envolvidas.
 - Exemplo: o conceito *mamífero* é mais genérico que o conceito *ser humano*.
 - Exemplo: o conceito *carro* é mais específico que o conceito *veículo*.
- Esse é o chamado ***relacionamento de herança***.
 - relacionamento de generalização/especialização
 - relacionamento de gen/espec

Generalizações e Especializações

- Terminologia
 - *subclasse X superclasse.*
 - *supertipo X subtipo.*
 - *classe base X classe herdeira.*
 - classe de *especialização* X classe de *generalização*.
 - *ancestral e descendente* (herança em vários níveis)
- Notação definida pela UML

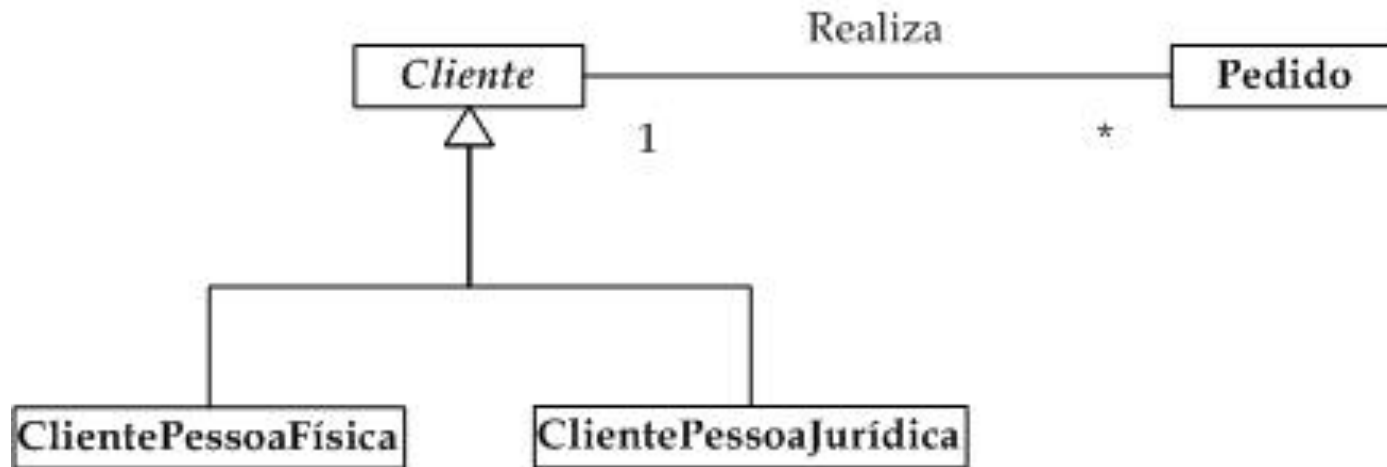


Semântica da Herança

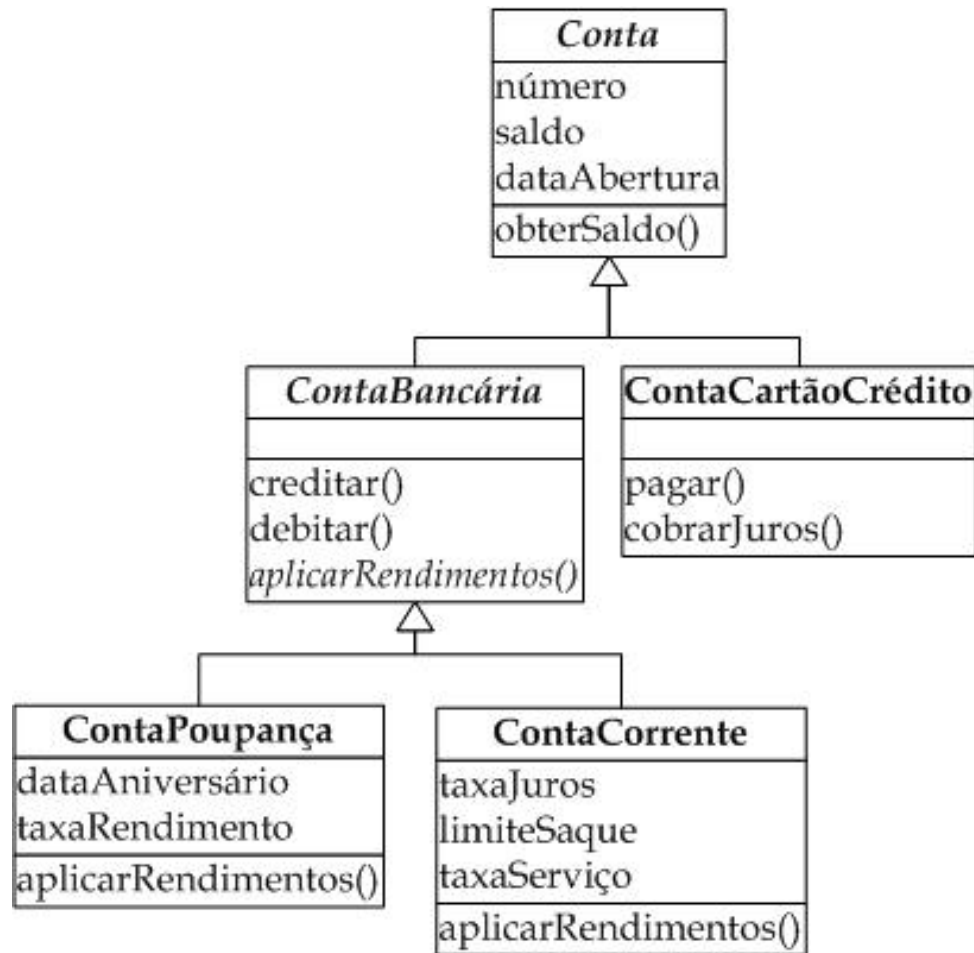
- Subclasses herdam as características de sua superclasse
 - É como se as características da superclasse estivessem definidas também nas suas subclasses
- Note a diferença semântica entre a herança e a associação.
 - A primeira trata de um relacionamento entre classes, enquanto que a segunda representa relacionamentos entre instâncias de classes.
 - Na associação, objetos específicos de uma classe se associam entre si ou com objetos específicos de outras classes.
 - Exemplo:
 - Herança: “Gerentes são tipos especiais de funcionários”.
 - Associação: “Gerentes chefiam departamentos”.

Herança de Associações

- Não somente atributos e operações, mas também associações são herdadas pelas subclasses.
- No exemplo abaixo, cada subclasse está associada a Pedido, por herança.



Propriedades da Herança

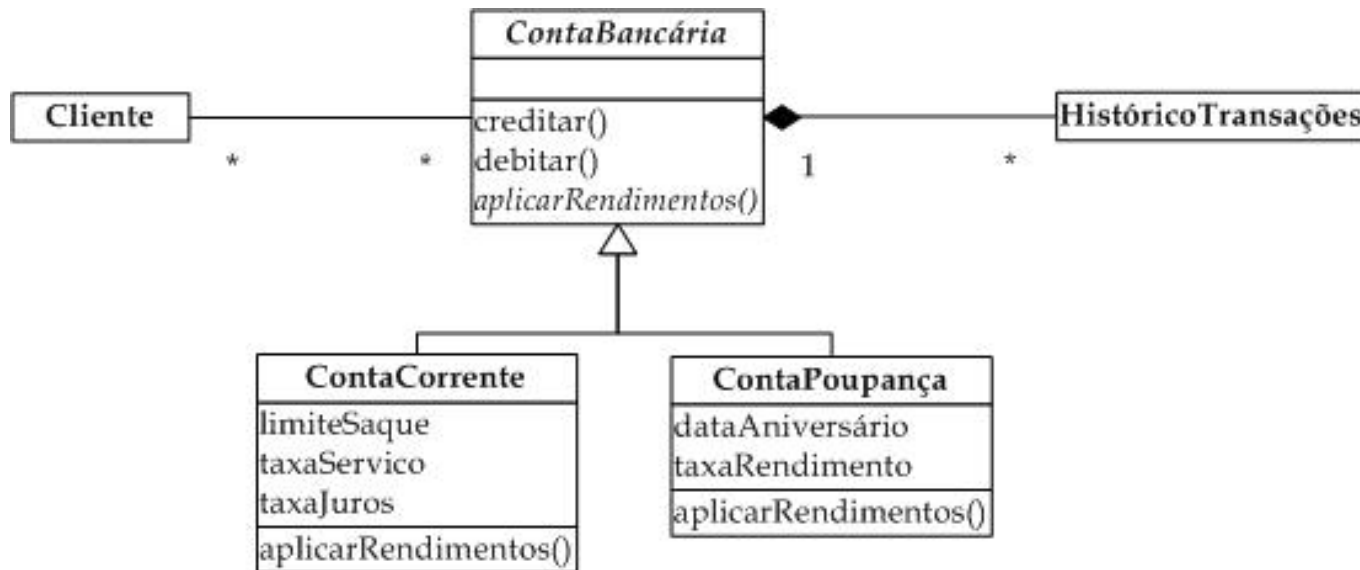


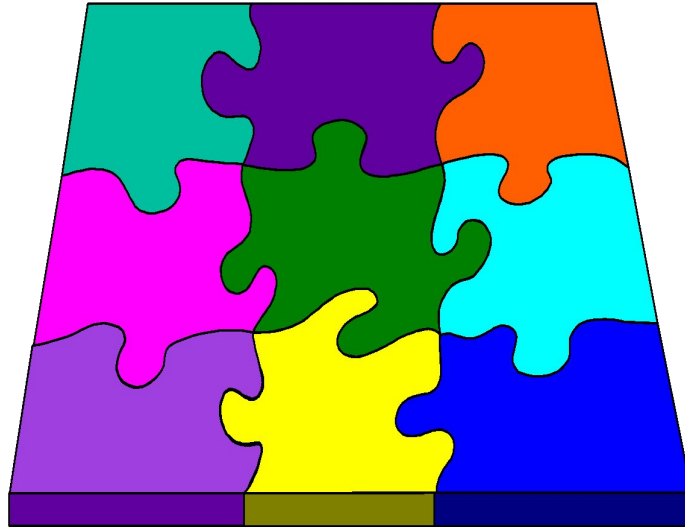
Classes Abstratas

- Usualmente, a existência de uma classe se justifica pelo fato de haver a possibilidade de gerar instâncias da mesma
 - Essas são as *classes concretas*.
- No entanto, podem existir classes que não geram instâncias diretas.
 - Essas são as *classes abstratas*.
- Classes abstratas são utilizadas para organizar e simplificar uma hierarquia de generalização.
 - Propriedades comuns a diversas classes podem ser organizadas e definidas em uma classe abstrata a partir da qual as primeiras herdam.
- Subclasses de uma classe abstrata também podem ser abstratas, mas a hierarquia deve terminar em uma ou mais classes concretas.

Notação para classes abstratas

- Na UML, uma classe abstrata é representada com o seu nome em *itálico*.
- No exemplo a seguir, ContaBancária é uma classe abstrata.





5.4 Técnicas para identificação de classes

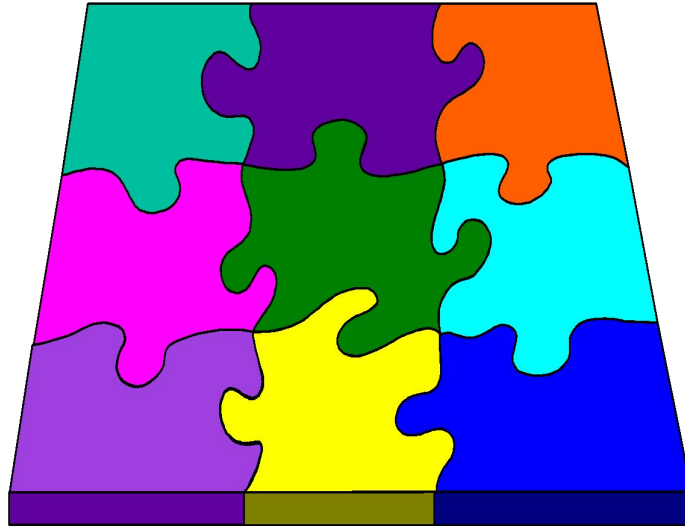
Apesar de todas as vantagens que a OO pode trazer ao desenvolvimento de software, um problema fundamental ainda persiste: identificar corretamente e completamente objetos (classes), atributos e operações.

Categorias de Conceitos

- Estratégia: usar uma lista de conceitos comuns.
 - **Conceitos concretos.** Por exemplo, edifícios, carros, salas de aula, etc.
 - **Papéis** desempenhados por seres humanos. Por exemplo, professores, alunos, empregados, clientes, etc.
 - **Eventos**, ou seja, ocorrências em uma data e em uma hora particulares. Por exemplo, reuniões, pedidos, aterrisagens, aulas, etc.
 - **Lugares:** áreas reservadas para pessoas ou coisas. Por exemplo: escritórios, filiais, locais de pouso, salas de aula, etc.
 - **Organizações:** coleções de pessoas ou de recursos. Por exemplo: departamentos, projetos, campanhas, turmas, etc.
 - **Conceitos abstratos:** princípios ou idéias não tangíveis. Por exemplo: reservas, vendas, inscrições, etc.

Análise de Casos de Uso

- Essa técnica é também chamada de identificação dirigida por casos de uso.
- Técnica preconizada pelo Processo Unificado.
- Nesta técnica, o MCU é utilizado como ponto de partida.
 - Premissa: um caso de uso corresponde a um comportamento específico do SSOO. Esse comportamento somente pode ser produzido por objetos que compõem o sistema.
 - Em outras palavras, a realização de um caso de uso é responsabilidade de um conjunto de objetos que devem colaborar para produzir o resultado daquele caso de uso.
 - Com base nisso, o modelador aplica a técnica de análise dos casos de uso para identificar as classes necessárias à produção do comportamento que está documentado na descrição do caso de uso.



5.5 Construção do modelo de classes

Construção do modelo de classes

- Após a identificação de classes, o modelador deve verificar a consistência entre as classes para eliminar incoerências e redundâncias.
 - Como dica, o modelador deve estar apto a declarar as razões de existência de cada classe identificada.
- Depois disso, os analistas devem começar a definir o mapeamento das responsabilidades e colaboradores de cada classe para os elementos do diagrama de classes.
 - Esse mapeamento resulta em um diagrama de classes que apresenta uma estrutura estática relativa a todas as classes identificadas como participantes da realização de um ou mais casos de uso.

Definição de propriedades

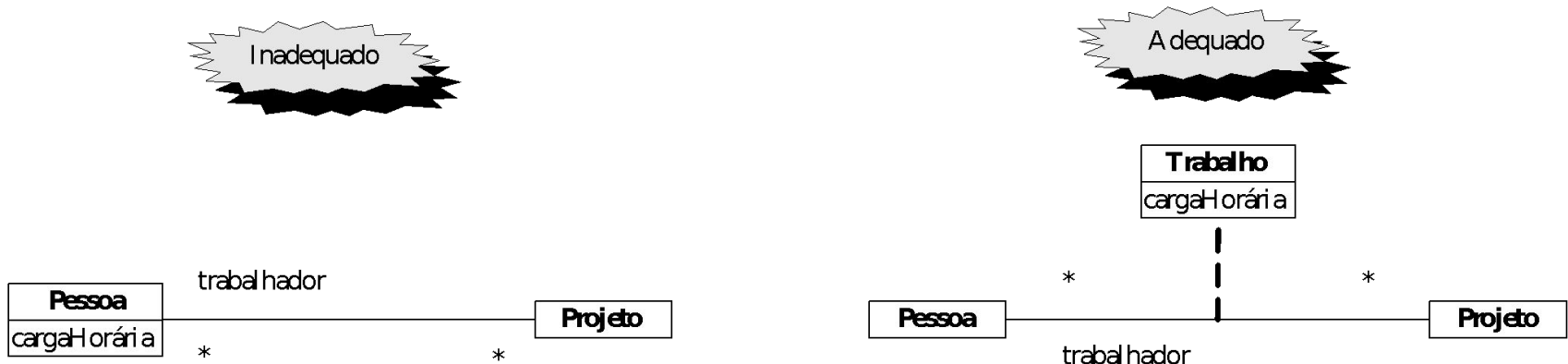
- Uma responsabilidade de conhecer é mapeada para um ou mais atributos.
- Operações de uma classe são um modo mais detalhado de explicitar as responsabilidades de fazer.
 - Uma operação pode ser vista como uma contribuição da classe para uma tarefa mais complexa representada por um caso de uso.
 - Uma definição mais completa das operações de uma classe só pode ser feita após a construção dos **diagramas de interação**.

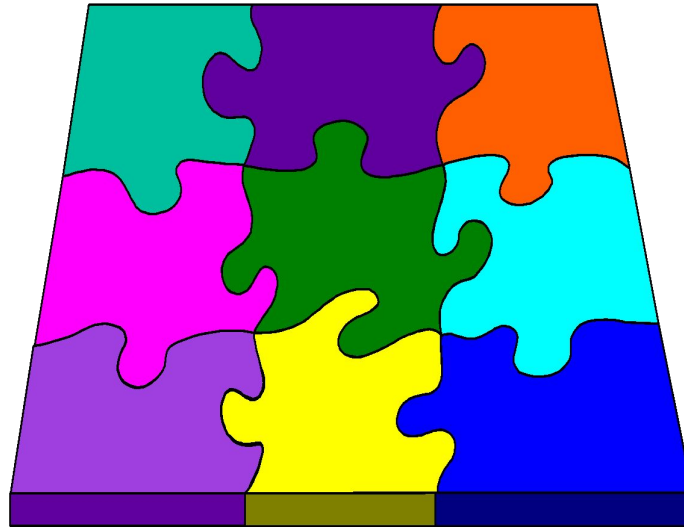
Definição de associações

- O fato de uma classe possuir colaboradores indica que devem existir relacionamentos entre estes últimos e a classe.
 - Isto porque um objeto precisa conhecer o outro para poder lhe fazer requisições.
 - Portanto, para criar associações, verifique os colaboradores de uma classe.
- O raciocínio para definir associações reflexivas, ternárias e agregações é o mesmo.

Definição de classes associativas

- Surgem a partir de responsabilidades de conhecer que o modelador não conseguiu atribuir a alguma classe.
 - (mais raramente, de responsabilidades *de fazer*)





5.6 Modelo de classes no processo de desenvolvimento

Modelo de classes no processo de desenvolvimento

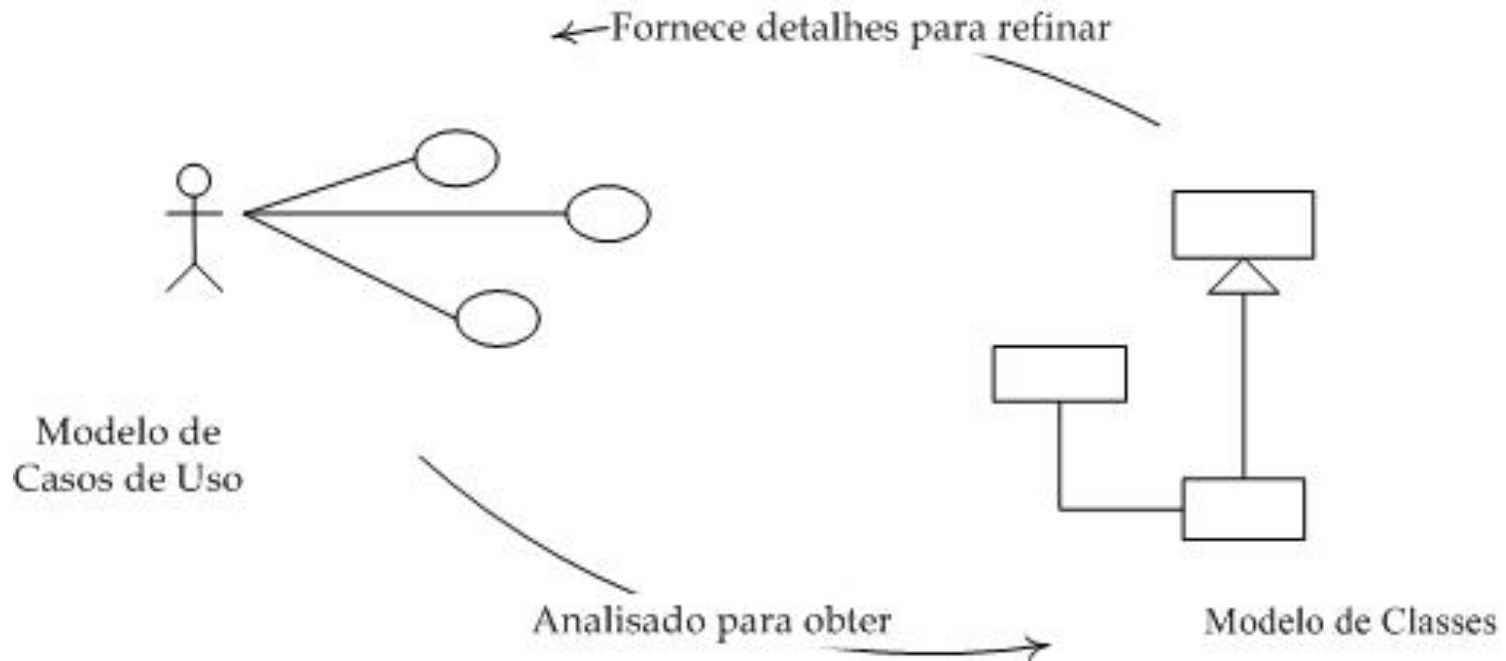
- Em um desenvolvimento dirigido a casos de uso, após a descrição dos casos de uso, é possível iniciar a identificação de classes.
- As classes identificadas são refinadas para retirar inconsistências e redundâncias.
- As classes são documentadas e o diagrama de classes inicial é construído, resultando no modelo de classes de domínio.

Modelo de classes no processo de desenvolvimento

- Inconsistências nos modelos devem ser verificadas e corrigidas.
- As construções do modelo de casos de uso e do modelo de classes são retroativas uma sobre a outra.
 - Durante a aplicação de alguma técnica de identificação, novos casos de uso podem ser identificados
 - Pode-se identificar a necessidade de modificação de casos de uso preexistentes.
- Depois que a primeira versão do modelo de classes de análise está completa, o modelador deve retornar ao modelo de casos de uso e verificar a consistência entre os dois modelos.

Modelo de classes no processo de desenvolvimento

- Interdependência entre o modelo de casos de uso e o modelo de classes.



Exercício rápido

- Modele o diagrama de classes de um sistema de mensagens instantâneas semelhante ao WhatsApp. O sistema deve permitir cadastro de usuários, envio de mensagens em conversas privadas e em grupos, armazenamento de histórico, administração de participantes em grupos e notificações de novas mensagens. Considere que existem diferentes tipos de mensagem, como texto, imagem, vídeo e áudio.